AI-Powered Sentiment Analysis System

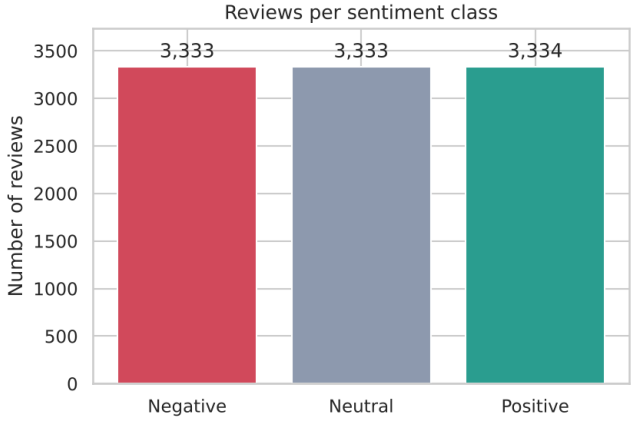
Evaluation Report · Month 2 Project · NLP + Machine Learning

1. Summary

A text classifier was built that labels customer reviews as **Positive, Negative or Neutral**. The final model is **Logistic Regression** on **TF-IDF** features (unigrams + bigrams, C = 10). On reviews containing sentences it had never seen it reaches **92.1% accuracy** (macro F1 0.921, macro ROC-AUC 0.984). On 90 freshly hand-written reviews accuracy is lower, **72.2%**, because the training vocabulary is very small. The model is saved as one scikit-learn pipeline and served through a Streamlit app.

2. Dataset and an important finding

The CSV has 10,000 rows and no missing values or duplicated rows. Classes are balanced (3,334 / 3,333 / 3,333). **However only 300 distinct review texts exist (100 per class), built from just 72 distinct sentences**, and no sentence appears under two labels. A normal random 80/20 split therefore puts the same texts in train and test and scores **100%** (100% of the distinct test texts were also in the training set). That number says nothing about real-world performance. Category and source are statistically independent of sentiment (chi-square tests), so only *review_text* is used.



Evaluation protocol used instead. Whole sentences are held out: 6 of 24 sentences per class form the test set (3,925 reviews), the other 6,075 reviews are for training, and every test review contains at least one sentence never seen in training. Cross-validation uses the same idea (3 sentence-grouped folds). For comparison, ordinary stratified 5-fold CV reports macro-F1 1.000, the grouped CV reports 0.854 for the same model.

3. Text preprocessing

Lower-casing; expanding contractions (didn't → did not); removing punctuation and digits; removing stop-words **except negations** (not, no, never…) because they flip meaning; dropping one-letter tokens. Stemming/lemmatisation was not used: the vocabulary is already tiny and unstemmed words keep the app's explanations readable. Ablation (leak-free CV macro-F1): A) 0.817; B) 0.795; C) 0.854. Keeping negations gave the best CV score; the gain is modest relative to fold-to-fold variation.

4. Feature extraction

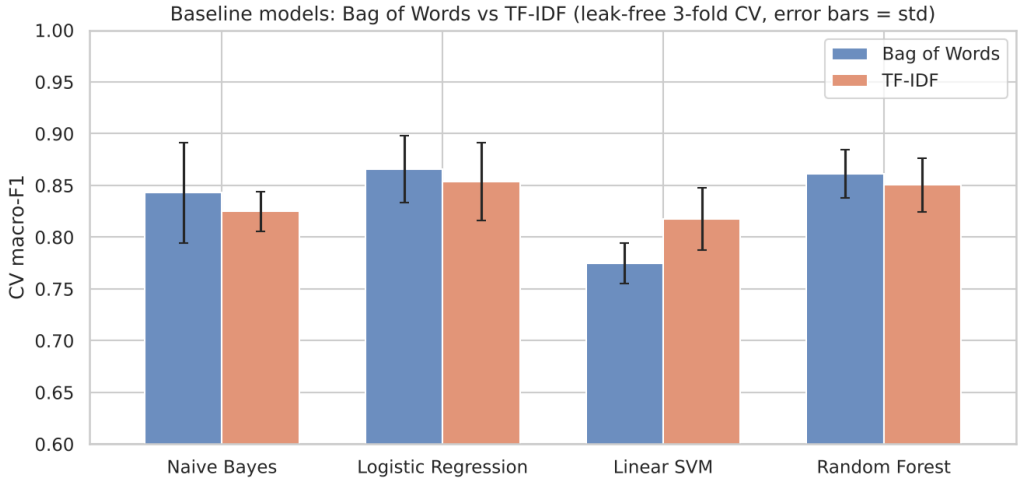
Bag of Words (word counts) and TF-IDF (counts down-weighted for words common to many reviews, length-normalised) were both tested, with unigrams or unigrams+bigrams. Bigrams such as 'not good' capture

a little word order and negation. The vectoriser type was treated as a hyper-parameter.

5. Model comparison (baseline, default parameters)

Features	Model	CV F1	CV std	Test acc.	Test prec.	Test recall	Test F1
Bag of Words	Naive Bayes	0.843	0.048	0.843	0.853	0.843	0.844
Bag of Words	Logistic Regression	0.866	0.032	0.883	0.888	0.883	0.884
Bag of Words	Linear SVM	0.775	0.020	0.871	0.882	0.871	0.872
Bag of Words	Random Forest	0.862	0.023	0.752	0.774	0.753	0.745
TF-IDF	Naive Bayes	0.825	0.019	0.868	0.870	0.867	0.868
TF-IDF	Logistic Regression	0.854	0.038	0.902	0.904	0.902	0.903
TF-IDF	Linear SVM	0.818	0.030	0.903	0.906	0.903	0.904
TF-IDF	Random Forest	0.850	0.026	0.728	0.757	0.728	0.718

Precision/recall/F1 are macro-averaged over the three classes. CV = sentence-grouped 3-fold on the training part.



6. Cross-validation and Grid Search

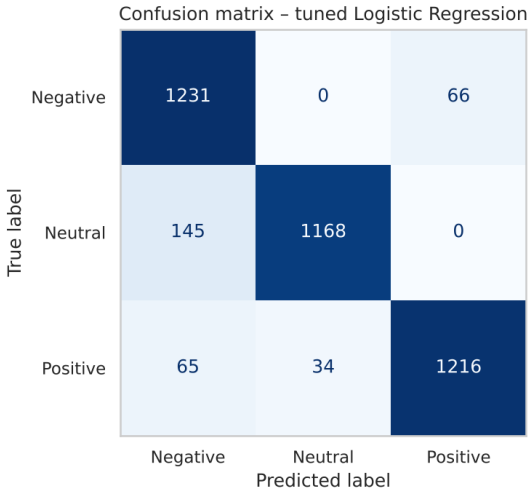
For each model family a grid search was run over vectoriser (BoW/TF-IDF), n-gram range and the main model parameters, scored by macro-F1 on the grouped folds.

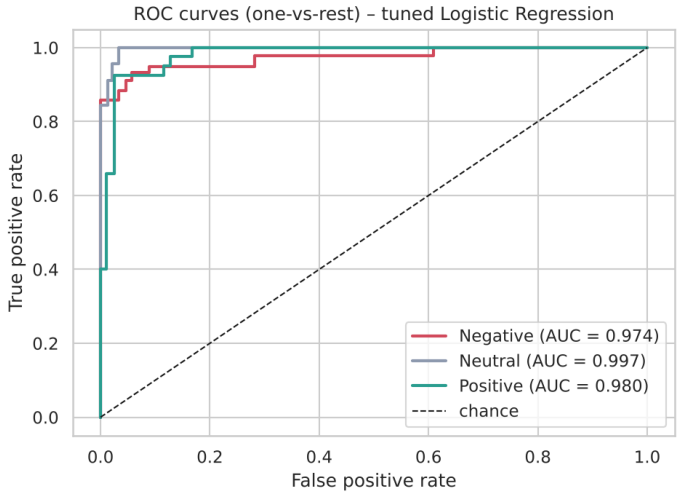
Model	Features	n-grams	Best params	CV F1 ± std	Test acc.	Test F1
Random Forest	Bag of Words	(1, 2)	{'max_depth': None, 'n_estimators': 100}	0.882 ± 0.015	0.790	0.784
Logistic Regression	TF-IDF	(1, 2)	{'C': 10}	0.880 ± 0.027	0.921	0.921
Linear SVM	TF-IDF	(1, 2)	{'C': 0.01}	0.874 ± 0.032	0.929	0.930
Naive Bayes	Bag of Words	(1, 2)	{'alpha': 0.1}	0.862 ± 0.042	0.875	0.876

Why Logistic Regression. The four tuned models' CV scores lie within about one standard deviation of each other, so they are statistically tied. Applying the one-standard-error rule, the simplest model that also gives usable class probabilities (needed for the app's confidence score) and interpretable weights was chosen: Logistic Regression. It also had the best or near-best test result among the tied models (Random Forest, nominally top in CV, generalised worst: 0.79 test accuracy).

7. Final model evaluation (held-out test, unseen sentences)

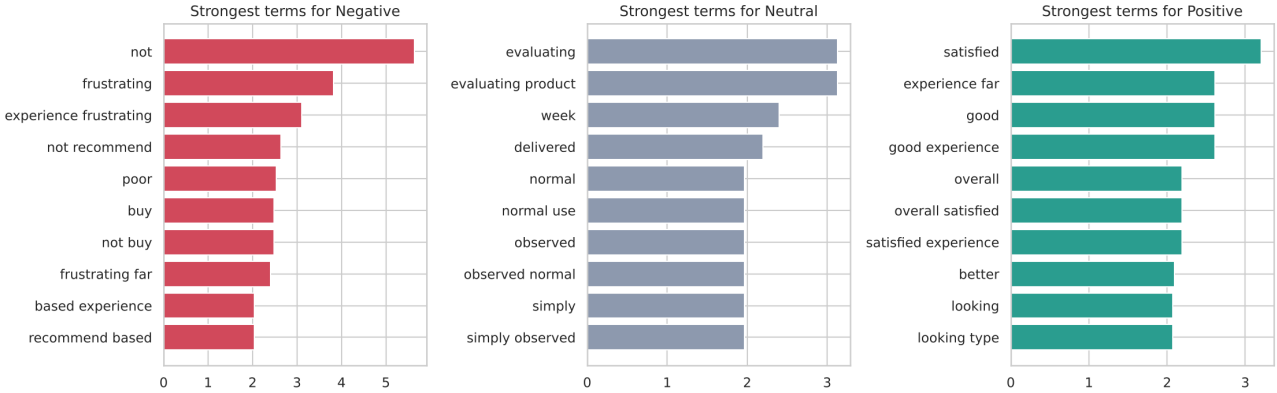
Accuracy **0.921** · macro precision **0.925** · macro recall **0.921** · macro F1 **0.921** · macro ROC-AUC **0.984** (per class: Negative 0.974, Neutral 0.997, Positive 0.980).





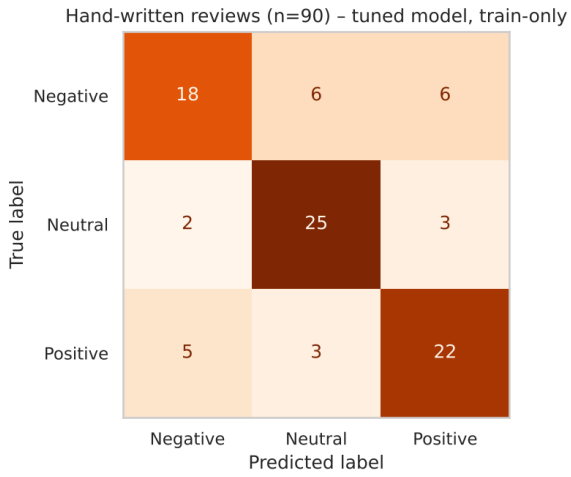
Metric	Baseline	Tuned final
Accuracy	0.883	0.921
Precision (macro)	0.888	0.925
Recall (macro)	0.883	0.921
F1 (macro)	0.884	0.921
ROC-AUC (macro)	0.967	0.984

Error analysis. About 8% of test reviews are wrong, and these come from a handful of sentences. A complaint containing positive-sounding words ("I expected much better quality for the price…") is predicted Positive with high confidence; neutral facts with unfamiliar words ("I purchased the item in black because that was the available color") drift to Negative. With only 18 unseen test sentences, one hard sentence moves accuracy by several points, consistent with the ±0.027 CV standard deviation.



8. Reality check on hand-written reviews

90 new reviews (30 per class, written by hand in everyday language; not used for training, tuning or selection). Train-only model: accuracy **72.2%**, macro F1 0.719. 59% of the words in these reviews never occurred in the training data. The deployed model (re-fitted on all 10,000 rows) scores 75.6%. This is the best indicator of real-world behaviour and it is clearly weaker than the in-dataset figures: a data limitation (72 sentences), not an algorithmic one.



9. Deployment

The final configuration is wrapped in one scikit-learn `Pipeline` (preprocessing + TF-IDF + Logistic Regression), re-fitted on all rows and saved with joblib to `models/sentiment_pipeline.pkl`. The Streamlit app loads it once (cached) and calls `predict_proba`, displaying the label, confidence, per-class probabilities and the words that drove the prediction. A low-confidence warning appears below 60%.

10. Limitations and future work

(1) Tiny, repetitive training data limits vocabulary coverage. (2) Sarcasm, mixed opinions and negation beyond simple patterns are not handled well. (3) English only. (4) Probabilities are not calibrated and can be confidently wrong. Next steps: train on real varied reviews, try lemmatisation and character n-grams, calibrate probabilities, add an explicit 'uncertain' outcome, and compare against a pre-trained transformer as a stretch goal.